

Bubble Sort Algorithm in Java

1 Introduction

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted.

2 Java Implementation

Below is the Java code implementation of the Bubble Sort algorithm:

```
1 public class BubbleSort {
2     public static void bubbleSort(int[] arr) {
3         int n = arr.length;
4         for (int i = 0; i < n - 1; i++) {
5             for (int j = 0; j < n - i - 1; j++) {
6                 if (arr[j] > arr[j + 1]) {
7                     // swap arr[j+1] and arr[j]
8                     int temp = arr[j];
9                     arr[j] = arr[j + 1];
10                    arr[j + 1] = temp;
11                }
12            }
13        }
14    }
15
16    public static void main(String[] args) {
17        int[] arr = {64, 34, 25, 12, 22, 11, 90};
18        bubbleSort(arr);
19        System.out.println("Sorted array:");
20        for (int i = 0; i < arr.length; i++) {
21            System.out.print(arr[i] + " ");
22        }
23    }
24 }
```

3 Time Complexity

The time complexity of Bubble Sort is:

- **Worst and Average Case:** $O(n^2)$, where 'n' is the number of items being sorted. This occurs when the array is in reverse order.
- **Best Case:** $O(n)$, when the array is already sorted.

A visual representation of the time complexity is shown in the graph below:

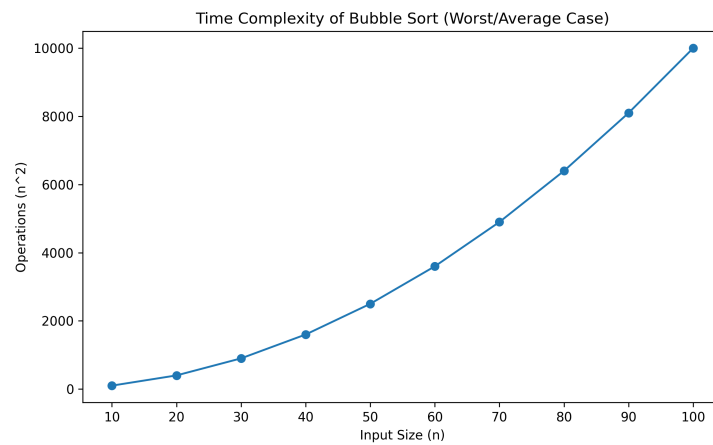


Figure 1: Time Complexity of Bubble Sort

4 Common Time Complexities

Here's a table differentiating common time complexities:

Notation	Name	Description
$O(1)$	Constant	Execution time is constant regardless of input size.
$O(\log n)$	Logarithmic	Execution time grows logarithmically with input size.
$O(n)$	Linear	Execution time grows linearly with input size.
$O(n \log n)$	Linearithmic	Execution time grows proportionally to $n \log n$.
$O(n^2)$	Quadratic	Execution time grows quadratically with input size.
$O(2^n)$	Exponential	Execution time doubles with each addition to the input.
$O(n!)$	Factorial	Execution time grows factorially with input size (extremely slow).

5 Conclusion

Bubble Sort is a simple algorithm to understand and implement, but it is inefficient for large datasets due to its quadratic time complexity.